

Midterm Project

Jordan Mckoy

Miguel Diaz-Alvarez

AJ Carroll

July 19, 2026

1 Overview

In this project, we were tasked with using the breast cancer dataset to implement and evaluate at least five classifiers from the following list:

- Logistic Regression
- Naive Bayes
- Perceptron
- Support Vector Machine
- Decision Tree Classifier
- K-Nearest Neighbors

2 Preprocessing

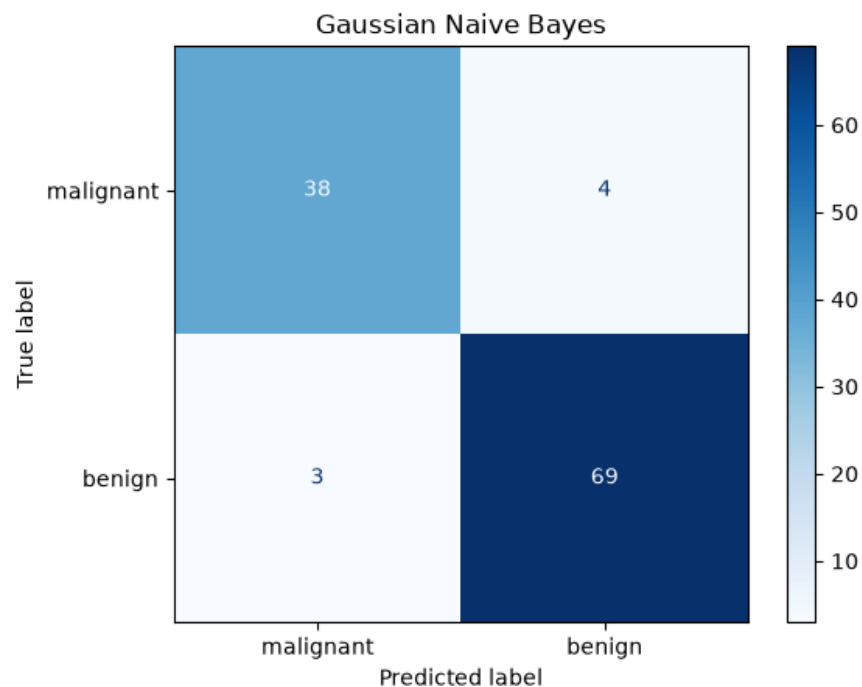
In the processing step, we split the data into 80 percent training and 20 percent testing. When splitting data, we set the stratify parameter to yes. This feature ensures that, in each split, classes that are rare, such as a positive cancer diagnosis, occur with the same frequency as in the main data set. We also opted to test different scalers (MinMax, Standard, no scaler). We did this by using Pipeline and GridSearchCV to test multiple different parameters at once.

3 Helper Functions

We used three helper functions in our notebook: `evaluate`, `summarize_grid`, and `weighted_recall`. The “`evaluate`” function takes the model and its name as input parameters. The function trains, tests, displays the confusion matrix, prints the classification report, and returns the model metrics. The “`summarize_grid`” function is used to compare different scalers. One of the questions asked how different scaling methods affect different algorithms; this function allows us to take a closer look at this. The “`weighted_recall`” function takes model predictions, true values, the weights of malignant and benign recall, and the weight of benign recall as input parameters. This function then computes a “weighted recall.” Since this is a medical context, we decided to emphasize false negatives, as they can lead to patients being left untreated. But if we only focused on that, that model would then diagnose more people with cancer to ensure no one is missed. To fix this, we used a portion of the malignant and benign recall scores from a default 60–40 split to obtain a more balanced model. This weighted recall metric was used to select the best model.

4 Naive Bayes

The Naïve Bayes didn’t have many parameters to test. There are different types of Naïve Bayes algorithms. We chose the Gaussian Naïve Bayes (GNB) algorithm and tested our three different scalers. The best scaler for GNB turned out to be no scaler at all. After doing some research on GNB, it turns out they are theoretically scale-invariant, so in practice, scaling would have little effect on the outcome.

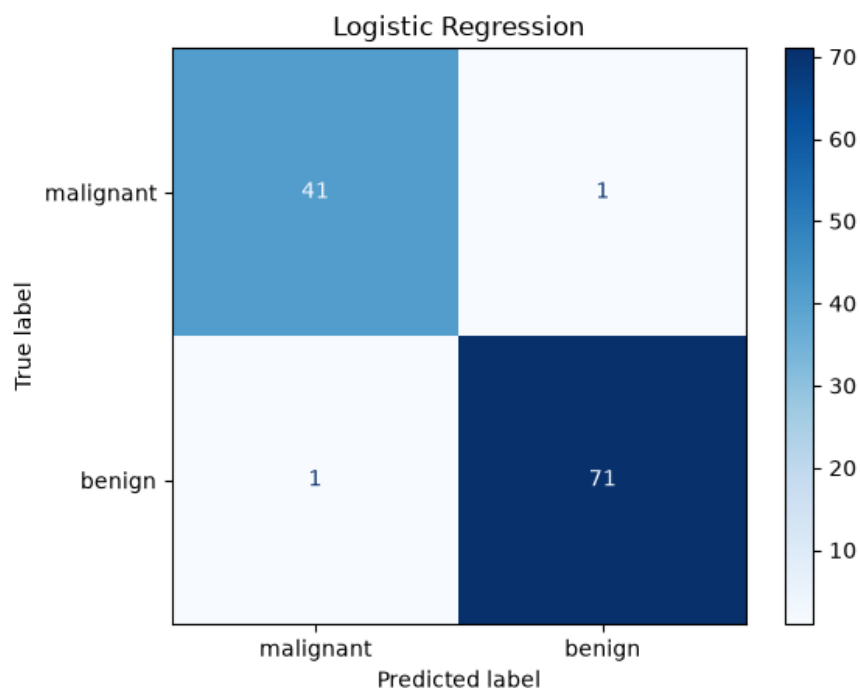


Confusion matrix for Naive Bayes model.

5 Logistic Regression

Logistic regression fits a linear decision boundary by modeling the log-odds of the benign class as a linear function of the features, with L2 regularization controlled by the inverse regularization strength C . We searched $C \in \{0.01, 0.1, 1, 10, 100\}$, the `liblinear` and `lbfgs` solvers, and both standard and MinMax scalers.

The best configuration was $C = 1$ with the `liblinear` solver and `StandardScaler` (CV score 0.9767). On the validation set it is the strongest model overall: **accuracy 0.9825**, precision 0.9861, recall 0.9861, F1 0.9861, and malignant recall 0.9762. It makes only two errors: one missed malignant case and one false alarm.



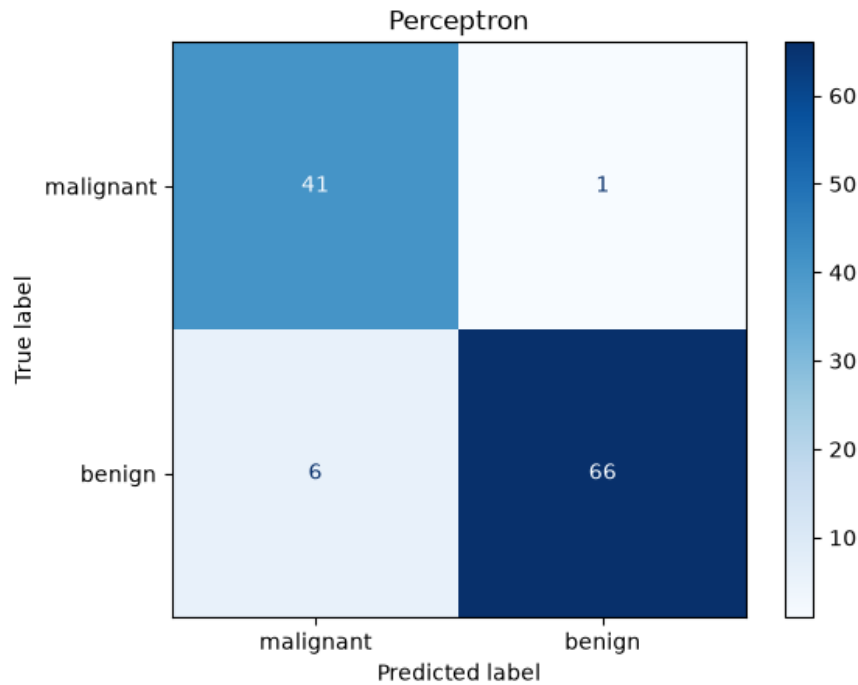
Confusion matrix for Logistic Regression model.

Scaling was essential here: gradient-based solvers converge poorly when features differ by four orders of magnitude, and standardization also makes the regularization penalty treat all coefficients comparably.

6 Perceptron

For the Perceptron model, we tested the penalty, alpha, max iterations, and scaler. The penalty parameter controls the type of regularization used by the model. We tested no penalty, l1, l2, and elasticnet. The alpha value controls how strongly the regularization is applied. The max iterations parameter controls how many times the model is allowed to update its weights before

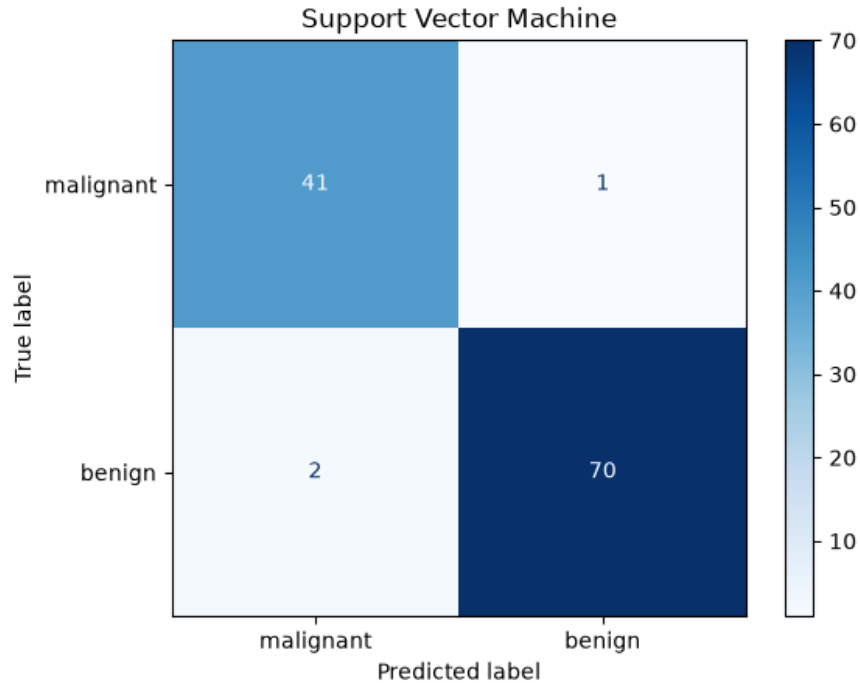
stopping. Since Perceptron is a linear model that updates weights based on classification mistakes, scaling is important because features with larger numeric ranges can have too much influence on the model. The grid search selected elasticnet, an alpha of 0.0001, a max iteration value of 1000, and StandardScaler as the best combination. This model had strong malignant recall, which means it was good at catching cancer cases. However, its benign recall was lower than some of the other models, meaning it had more false positives. In this project, that tradeoff is still important to consider because false negatives are more serious, but too many false positives can also lead to unnecessary follow-up testing.



Confusion matrix for Perceptron model.

7 Support Vector Machine

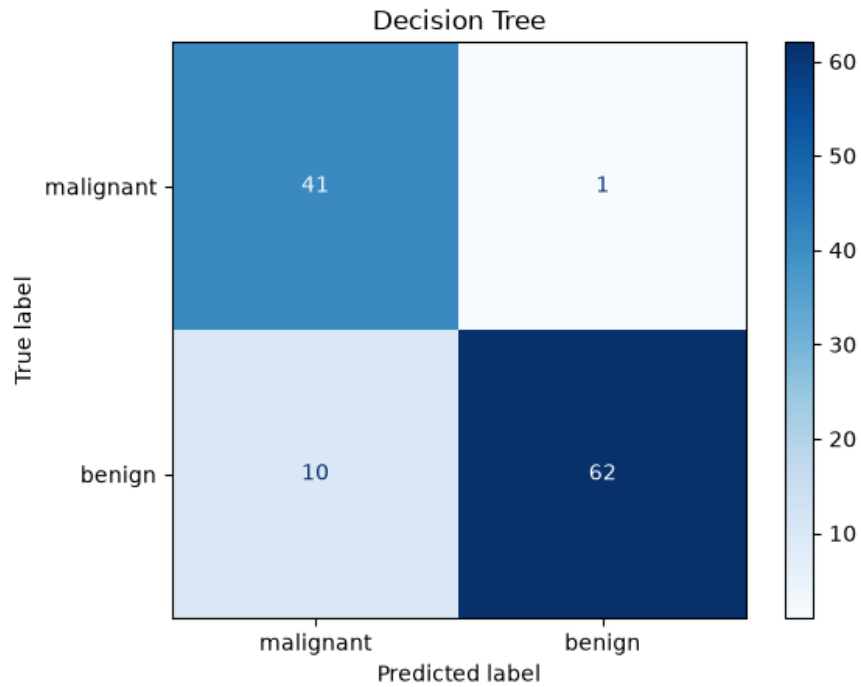
Support Vector Machine (SVM) had multiple parameters to test: kernel, C, gamma, and scaler. The kernel controls the type of decision boundary the model can create. A linear kernel creates a straight boundary, while the rbf kernel can create a more flexible curved boundary. The C parameter controls how much the model penalizes classification mistakes. A smaller C allows a wider margin with more mistakes, while a larger C tries harder to classify the training data correctly. The gamma parameter controls how far the influence of each training sample reaches when using the rbf kernel. We also tested different scalers because SVM is sensitive to feature scale, especially when using distance-based kernels such as rbf. The grid search selected an rbf kernel, a C value of 10, gamma set to scale, and MinMaxScaler as the best combination. This model performed very well overall and was one of the models tied for the highest malignant recall. It also kept strong accuracy, precision, recall, and F1-score, which made it one of the best balanced classifiers in the project.



Confusion matrix for Support Vector Machine model.

8 Decision Tree Classifier

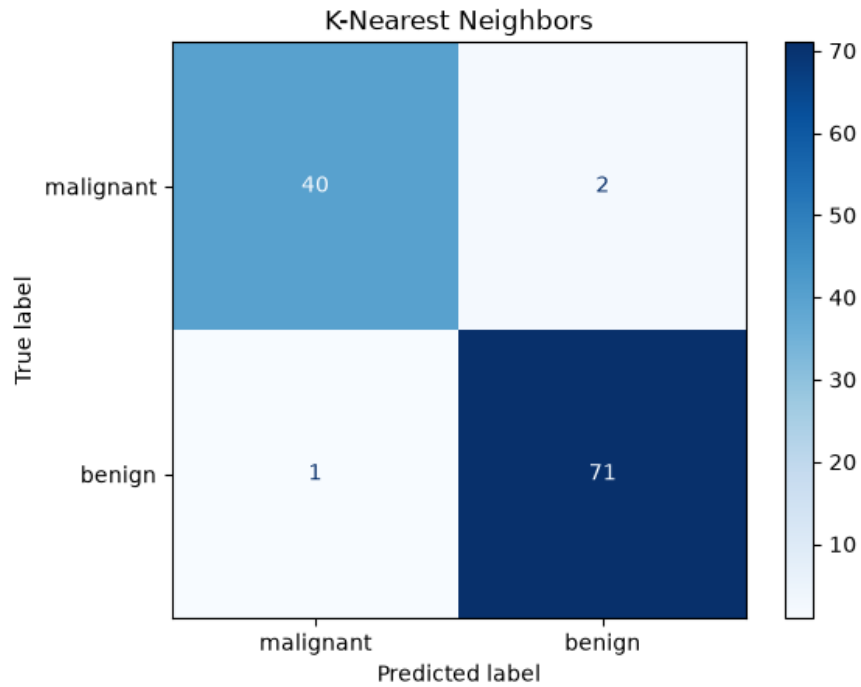
Decision Tree Classifier (DTC) had quite a lot of parameters to test: criterion, max depth, ccp alpha, min samples leaf, and class weight. The criterion is the function used to measure the quality of a split. Max depth is the maximum depth of the tree. CCP alpha is a complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than ccp alpha will be chosen. The min samples leaf is the minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least min samples leaf training samples in each of the left and right branches. The class weight parameter controls the weights associated with each class. When set to balanced, the weights are adjusted inversely proportional to the class frequencies in the training data, so the minority class (malignant) counts more heavily when the tree evaluates splits. When left at the default, all samples are weighted equally. We included this parameter because the dataset is imbalanced and, in this medical context, we wanted to test whether penalizing missed malignant cases at training time would improve malignant recall. The grid search selected gini, a max depth of none, a ccp alpha of 0.0, a min samples leaf of 20, and default class weights as the best combination. One note on DTC scaling: it had no effect on the outcome during initial testing, so the test was removed to speed up runtime.



Confusion matrix for Decision Tree Classifier model.

9 K-Nearest Neighbors

For K-Nearest Neighbors (KNN), we tested the number of neighbors(`n_neighbors`), weights, and scalers. The best-scoring model had the following parameters: 7 neighbors, uniform weights, and `MinMaxScaler`.



Confusion matrix for K-Nearest Neighbors model.

10 Comparison of Models

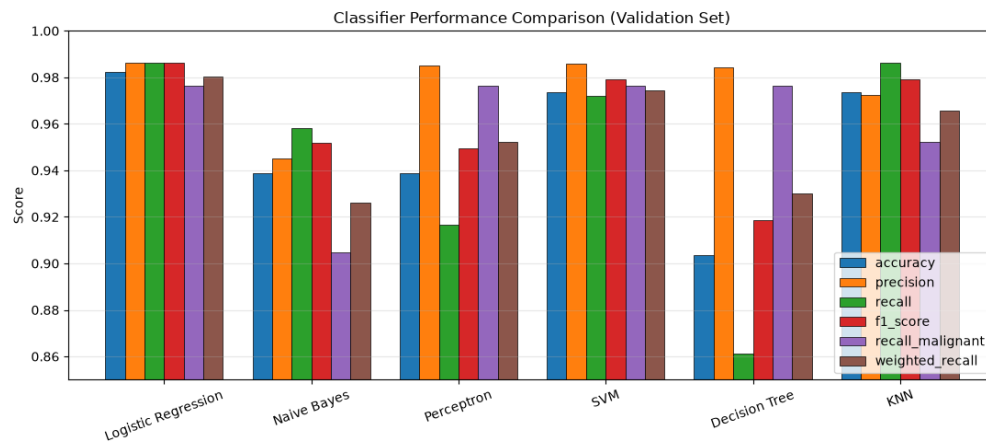


Chart comparing the metrics of the different models.

Table 10.1: Model performance comparison on the test set.

	Accuracy	Precision	Recall	F1	Recall (Mal.)	Weighted Recall
Logistic Regression	0.9825	0.9861	0.9861	0.9861	0.9762	0.9802
SVM	0.9737	0.9859	0.9722	0.9790	0.9762	0.9746
KNN	0.9737	0.9726	0.9861	0.9793	0.9524	0.9659
Perceptron	0.9386	0.9851	0.9167	0.9496	0.9762	0.9524
Decision Tree	0.9035	0.9841	0.8611	0.9185	0.9762	0.9302
Naive Bayes	0.9386	0.9452	0.9583	0.9517	0.9048	0.9262
